

无痛学串口

PB22111665 胡揚嘉

串口的基础理解

串行通信：是指通过一条通信线，将数据按位依次传输，每次传输一个数据位。每个数据位在时间上占据一个固定的时间长度，通常称为“波特率”或者“比特率”。串行通信的优势在于它的硬件实现简单，仅需少数几根信号线，尤其适用于远距离或低速的数据交换，比如计算机与计算机、计算机与外设之间的通信。

关键词：逐位传输 + 固定时间间隔

1. 单周期序列生成与检测器：负责根据波特率生成和检测每个数据位的时序信号，确保数据在规定的时间内传输。
2. 分频器(广义)：根据时钟频率，匹配所需的波特率，从而保证数据传输的同步。

所以结论：

1. 串口 = 单周期序列检测与生成器 + 分频器
2. 序列检测与生成器 = 序列检测与生成器本身 + 隐含的状态机思想

序列生成与检测器 通常依赖于一个隐含的状态机逻辑。状态机帮助管理数据传输的各个阶段，包括起始位、数据位以及停止位的时序，从而确保数据在发送与接收过程中保持正确的同步。

下面我们逐个了解

序列检测与生成器(单周期)

我们不妨给出一类题目：

Case1

每收到一个 `valid` 信号，按照每个时钟一位的速率生成长度为8的序列，序列为"10101010"，输出完毕给出 `ready` 信号表示一次输出结束。

很好实现对吧 😊

这里我们实现了固定序列的生成

Case2

每收到一个 `valid` 信号，当周期保存传入的8位序列，在接下来的时钟周期内，按照每个时钟一位的速率输出这个序列，输出完毕给出 `ready` 信号表示一次输出结束。

这里我们唯一的不同就是我们需要保存传入的序列，然后输出的序列是由传入的序列决定的。

这里我们实现了指定序列的生成 😊

看看与串口输出的差别在哪里？

1. 没有起始位和结束位
2. 串口不是每一个时钟传输一次（留到后面讲解）

Case3 进阶

不限定长度的序列怎么处理？有校验位的序列怎么处理？留待同学们自行学习提升思考 😓

好了，现在我产生了序列，需要检测序列了。不妨假设序列都来自于上面产生的序列，而且我们 凭空 制造了起始位

我们不妨再做一个题目：

已知每个周期可以读取一个数，检测输入的8位序列是什么

怎么做？

当接收到第一个“下降沿”时，我们开始采集接下来的8个时钟周期的数据。将这些数据拼接起来，形成一个完整的8位序列。

OK，我们已经完成了序列检测（bushi 😊）

还有什么要注意的？

1. 阅读全文后，给出真正的串行输出单元与这里的序列检测器的区别，留待各位作为实验报告的思考，建议100字以内，清晰明确。

两个频率的恩怨

串行通信协议

常见的串行通信协议包括RS-232、RS-485、UART等。这些协议规定了数据传输的格式、波特率、起始位、停止位、校验位等参数。在串行口通信中，数据被分割成多个数据位，每个数据位逐个传输。数据位之间通过特定的时钟信号进行同步。发送端将数据位按照协议规定的格式发送到传输线上，接收端通过解析接收到的数据位来恢复原始数据。

我们注意到：**波特率**，它与时钟频率有什么关系呢？

读读波特率的定义：

波特率（bandrate），指的是串口通信的速率，也就是串口通信时每秒钟可以传输多少个二进制位。比如每秒钟可以传输9600个二进制（传输一个二进制位需要的时间是1/9600秒，也就是104us），波特率就是9600。

也就是说，我们接收、传输一个序列的比特位的时间变成了 $T_{band} = \frac{1}{f_{band}}$

但是上面我们假设的是每一个时钟周期一次呀？

怎么办？

你说呢？

计时器数呀！

$$Cnt_{fact} = \frac{T_{band}}{T_{clk}} = \frac{f_{clk}}{f_{band}}$$

这表示，每传输一个数据位，时钟信号需要经过 Cnt_{fact} 个时钟周期。在接收端，计时器会用这些时钟周期来对每一个数据位进行采样，从而实现数据的同步接收。

注意：

Attention: 禁止使用分频器的输出作为时钟信号！！ 严厉禁止

发送数据的时钟控制

在发送数据时，在发送每一位数据时，单个比特位的维持周期需要与商定的通信协议的波特率相同，即每个比特的发送时长是一个较长的时间。

接收中的去抖动问题

在接收数据时，由于时钟和信号波形的不匹配，可能会导致接收到的信号不稳定，从而出现误码的情况。为了避免这种情况，接收端会利用多个采样点来确认每个数据位的值，尤其是在开始位和停止位附近的时刻。

计时器（或者说采样点）帮助接收端将每个比特的发送时机与接收到的信号进行匹配和对比，减少误差。在实际的串口通信中，接收端会对每个数据位进行多次采样，然后通过比较多次采样的结果来确定数据位的值。

去抖动可以通过以下几种方法来实现：

1. **多次采样**：在接收时使用多个采样点，尤其是对每个数据位进行多次采样（例如，使用三个采样点）。通过比较多个采样点的结果来减少由于抖动引起的误差。
2. **中心采样**：在每个数据位的中间采样。这样可以确保每个数据位的采样时机比较稳定，尤其在信号的不稳定或噪声较多的情况下。

在我们的实验中，建议对起始位开始，到起始位中间的所有信号进行检测(去抖动)，确认不变后，认为之后到终止位前面的信号都是稳定的，在 **每一个比特位数据的中心** (我们的采样点)进行采样作为我们的结果。

为什么TA要像上面一样规定：

因为更简单，去抖动和检测在这次实验并不是我们想要强调的。而且想想对应的发送数据的逻辑，大部分情况下能够保证是正确的。

总结而言：

1. 对于发送端而言，每个数据位的维持时间是固定的，与波特率紧密相关，确保数据按预定的速率发送。而接收端通过利用计时器，确保在每个数据位的正确时机采样。通过对波特率和时钟频率的协调，可以有效地同步发送和接收的数据流，确保在波特率不变的情况下，数据传输能够顺利进行。
2. **维护两个频率的不同**：需要通过计时器或其他手段完成同步，确保数据传输的准确性。这里的关键是时钟频率必须足够高，以满足波特率的要求。
3. **读取中的去抖动问题**：通过多次采样、中心采样等方法，可以减少由于信号噪声或时钟偏差导致的误差。

状态机思想

为什么要提这个？

你说得对，实际上在串口通信中，并不一定非要用状态机来完成所有任务，但状态机的思想在很多情况下确实能**简化问题**和提高系统的**可靠性**。

为什么要引入状态机？

状态机的核心思想是将一个复杂的过程划分为一系列独立的**状态**，并根据输入（例如接收到的比特或字节）来进行状态的转换。每个状态负责完成一部分工作，而状态之间的转换由外部条件或事件触发。将串口通信问题转化为状态机模型，可以帮助我们更加清晰地理解数据的传输、接收以及错误处理等环节的工作流程。

状态机在串口通信中的作用

1. 处理数据帧的接收和发送

串口通信通常会以**数据帧**为基本单位进行数据交换，每一帧的数据包括起始位、数据位、校验位和停止位。状态机能够清晰地控制每个数据帧的处理过程，比如：

- **接收端**的状态机可以从接收到的起始位开始，逐步进入各个状态，直到完成数据位的接收、校验位的处理和停止位的检查。
- **发送端**的状态机则会按照波特率等参数逐个比特地发送数据，并且在发送完一帧数据后返回初始状态，准备发送下一帧。

通过状态机模型，接收和发送的每个过程都被**分解**成了清晰的状态，避免了处理逻辑的混乱和出错的可能。

将复杂的整块逻辑切分成

1. 较小的，有相同特性和处理方法的子逻辑块——一个状态
2. 各个子状态之间的转变关系——如果能够抽象出子状态的含义，实际上对程序员也更友好

2. 简化复杂逻辑

在一些复杂的串口协议中，通信双方可能需要进行多次的数据交互，甚至可能涉及到一些控制字符的处理。状态机可以帮助我们将这些复杂的交互拆解成简单的状态和状态转换。每个状态只处理一个小的任务，而多个状态的组合则完成了整个通信流程。

例如，在一些**握手协议**（如XON/XOFF）或**流控制**（如RTS/CTS）中，状态机通过不同的状态转移来控制数据的传输，避免了数据丢失或溢出的情况。

3. 定时和超时管理

在串口通信中，定时和超时管理非常重要。状态机能够有效地控制超时检测和定时器的使用：

- 当接收数据时，状态机会跟踪时间，确保在规定的时间内完成数据接收。如果超时，则触发错误状态，重新启动接收过程。
- 对于发送端，状态机可以控制每个数据位、数据帧的发送时机，避免过快或过慢的问题。

通过这种状态驱动的定时机制，串口通信的过程可以高效且稳定地执行。

总结

状态机在串口通信中的作用，实际上就像是为整个通信过程划定了一条**清晰的路线图**。它让你可以在复杂的通信流程中，以模块化和有序的方式逐步处理每一个环节。尽管你不使用状态机也能完成串口通信，但在实际应用中，状态机能让你避免错乱、提升可维护性，并且在错误处理、超时管理等方面提供了更强的可控性。

下面是举例说明：

仅介绍UART_TX的情况：

简单分层

可以简单分为两个状态：空闲与工作

空闲：没有数据需要输出

工作：发送一个完整的数据。——具体的怎么发送就在内部完成，状态机设计的时候不管

我们需要关注：怎么进入 work 状态，退出 work 状态会有什么信号输出指示，具体的 work 内部应该设计

复杂分层

分为4个状态，实际上是将 work 状态进一步切分成等价类。

起始位、终止位的思想与数据位不一样，所以要分开处理。

代价：我们需要解决4个状态之间的转变关系和处理逻辑

以两个信号的处理为例

在TX中，我们需要确认接收方有 valid——准备好了，才能开始工作。

但是，假如不在IDLE状态——TX模块已经开始工作了，那么我们需要考虑接收方的 valid 吗？显然**不需要**

是的，我们只在IDLE状态才会有需要考虑 valid

我们的输出 ready 信号工作原理

注：ready信号告诉外界现在自己模块能否进行发送数据的功能

```
assign ready = (state == S_Stop || (state == S_IDLE && nextstate != S_start)) ?
1'b1 : 1'b0;
```

下面是经典状态机模型

```
localparam S_IDLE   = 4'b0001;
localparam S_START  = 4'b0010;
localparam S_DATA    = 4'b0100;
localparam S_STOP    = 4'b1000;
```

```

logic [ 3: 0] state;
logic [ 3: 0] next_state;

always @(posedge clk) begin
    if(!rstn) begin
        state <= S_IDLE;
    end
    else begin
        state <= next_state;
    end
end

always @(*) begin
    case (state)
        S_IDLE: begin
            if (start) begin
                next_state = S_START;
            end
            else begin
                next_state = S_IDLE;
            end
        end
        S_START: begin
            next_state = S_DATA;
        end
        S_DATA: begin
            if (stop) begin
                next_state = S_STOP;
            end
            else begin
                next_state = S_DATA;
            end
        end
        S_STOP: begin
            next_state = S_IDLE;
        end
        default: begin
            next_state = S_IDLE;
        end
    endcase
end

```

state只被rstn重置，剩余时间由next_state计算。

next_state由当前状态 state 和某些信号共同控制，决定下一个状态。

而通常使用state来表明当前处于什么状态，进而决定当下所有的 信号和寄存器 应该如何工作和转变

